

Práctica 1: Detección de un objeto plano

El enunciado corresponde a la primera práctica calificable: **entrega día del examen de mayo.**

Importante: En esta práctica solamente se podrán utilizar las técnicas de procesamiento de imagen vistas en clase. **No está permitido utilizar clasificadores** (módulos ml y dnn de OpenCV) ni otras técnicas parecidas (p.ej. paquetes de python como sklearn, torch, etc.).

1 Copias de código o de la memoria

El código desarrollado en las prácticas debe de ser original. La copia (total o parcial) de prácticas será sancionada, al menos, con el SUSPENSO en la misma. En estos casos, podrá significar, en la siguiente convocatoria y a discreción del profesor, el tener que **resolver nuevas pruebas y la defensa de las mismas de forma oral. Las sanciones derivadas de la copia, afectarán tanto al alumno que copia como al alumno copiado.**

Para evitar que **cuando se usa código de terceros** sea considerado una copia, se debe **citar siempre la procedencia** de cada parte de **código no desarrollada por el propio alumno** (con comentarios en el propio código y con mención expresa en la memoria de las prácticas). El plagio o copia de terceros sin la cita correspondiente (p.ej. una página web o a través de ChatGPT o similares) ya sea en el código a desarrollar en las prácticas y/o de parte de la memoria de las prácticas, acarrearán las mismas sanciones que en la copia prácticas de otros alumnos.

2 Detección de un objeto plano con un diseño particular

Se desea construir un sistema que permita la detección de un objeto plano (ver Figura 1), primero en la imagen y después en 3D con respecto a la cámara utilizada y de la que se conoce la matriz de intrínsecos.



Figura 1: Plantilla utilizada en la práctica (izq.) y una imagen de la misma impresa en papel (dcha.)

Observando la Figura 1 podemos hacernos una idea sobre el tipo de información que nos permitirá localizar el objeto plano en la imagen. Existe una zona de textura con diferentes colores, algunos elementos como tijeras, logos, etc. 6 cuadrados que se corresponden con un cubo desplegado y un marco de color rojo separado de la zona de textura con un margen blanco. Por tanto, para detectar el objeto plano podríamos utilizar:

- Emparejamiento de puntos de interés con descriptores y una homografía, H_t^{img} , como modelo geométrico ajustado con RANSAC (cv2.findHomography). La matriz H_t^{img} permitirá transformar coordenadas sobre la imagen de la plantilla (Figura 1 izquierda) a la imagen de entrada (Figura 1 derecha).
- Los píxeles que pertenecen a los rectángulos que enmarcan el marco rojo en el exterior del objeto. Si encontramos las esquinas exteriores del marco podremos calcular la homografía del plano con la imagen de la plantilla (Figura 1 izquierda).
- Cualquier otra técnica que tenga en cuenta el color y la forma de los patrones en la zona central (p.ej. líneas o color).

2.1 Desarrollo de la práctica

El objetivo de la práctica es localizar la posición y orientación en 3D del objeto plano con respecto a la cámara y dibujar sobre el plano un cubo en 3D en una posición determinada (hacer Realidad Aumentada). Podemos ver un ejemplo del resultado buscado en la Figura 2.



Figura 2: Ejes del sistema de referencia 3D sobre el plano (esquina superior izquierda de la plantilla) y cubo 3D proyectado en la posición deseada.

Para desarrollar la práctica se proporcionan diferentes imágenes del objeto plano tomadas con la misma cámara. Además, se proporciona la imagen de la plantilla recortada que se muestra en la Figura 1. Esta imagen se ha obtenido escaneando la hoja de papel en formato DIN A4 que aparece en las imágenes en las que se pretende localizar.

La práctica consistirá en resolver los siguientes subproblemas:

1. Encontrar los parámetros de la homografía (matriz 3×3), $\mathbf{H}_t^{\text{img}}$, que permite transformar las cuatro esquinas de la imagen de la plantilla (imagen *template_cropped.png*) en las correspondientes cuatro esquinas sobre la imagen de entrada (Figura 1 derecha).
2. Usando `cv2.getPerspectiveTransform`, encontrar la homografía, $\mathbf{H}_w^t$, que lleva desde las cuatro esquinas de la plantilla impresa en papel y que se miden en milímetros, a las cuatro esquinas de la imagen de la plantilla (imagen *template_cropped.png*). Es importante establecer las coordenadas en mm sobre la plantilla real de tal manera que todos los puntos sobre ella tengan $Z=0$, y que el eje Y va de izquierda a derecha y el X va de arriba a abajo. El origen del sistema de referencia, punto $X=Y=Z=0$, estará justo donde se ha dibujado el origen de coordenadas en la Figura 2. Para establecer las coordenadas reales sobre la plantilla tendremos que tener en cuenta las medidas que aparecen en la Figura 3.

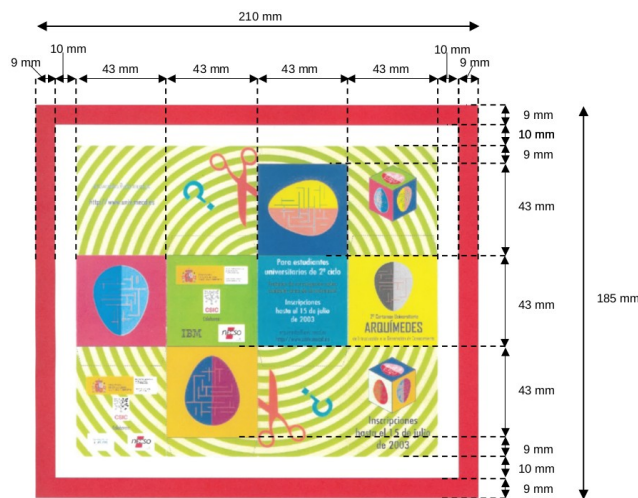


Figura 3: Medidas en milímetros tomadas sobre el objeto real (hoja impresa en DIN A4).

Al proyectar los ejes sobre la imagen de entrada estos deberán aparecer como en la Figura 4 (el eje Z en azul saliendo del plano, el Y creciendo hacia la derecha y el X hacia abajo en el plano).

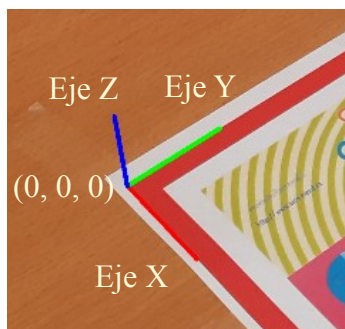


Figura 4: Detalle de los ejes del sistema de referencia 3D sobre el plano (esquina superior izquierda de la plantilla) y de las coordenadas del origen de coordenadas sobre el mismo.

3. Conocidas $\mathbf{H}_t^{\text{img}}$ y $\mathbf{H}_w^t$, podemos encontrar la homografía, $\mathbf{H}_w^{\text{img}}$, que va desde coordenadas en mm sobre objeto plano real a los puntos correspondientes sobre la proyección del mismo sobre la imagen. La forma de calcularla es la multiplicación de las dos matrices:

$$\mathbf{H}_w^{\text{img}} = \mathbf{H}_t^{\text{img}} \cdot \mathbf{H}_w^t$$

4. Con H_w^{img} y la matriz de intrínsecos de la cámara proporcionada con el material de la práctica, K , tal y como se explicará en clase, se podrá obtener la posición relativa de la cámara, R y t , con respecto al sistema de referencia colocado sobre el plano real. A partir de K , R y t se podrá obtener la matriz de proyección de la cámara, $P = K [R | t]$, posicionar objetos 3D sobre el plano real midiendo coordenadas en mm y permitiendo la visualización mediante su proyección sobre la imagen con P .

El esquema general de la geometría del problema se puede ver en la Figura 5. Como el objeto es plano y sus coordenadas tienen $Z=0$, se puede usar la homografía H_w^{img} transformado únicamente las coordenadas (X', Y') o, una vez conocida la R y t de la cámara con respecto al plano, se puede usar la matriz de proyección completa, P , para proyectar ya cualquier punto del espacio con sus coordenadas (X', Y', Z') , incluyendo los del objeto plano.

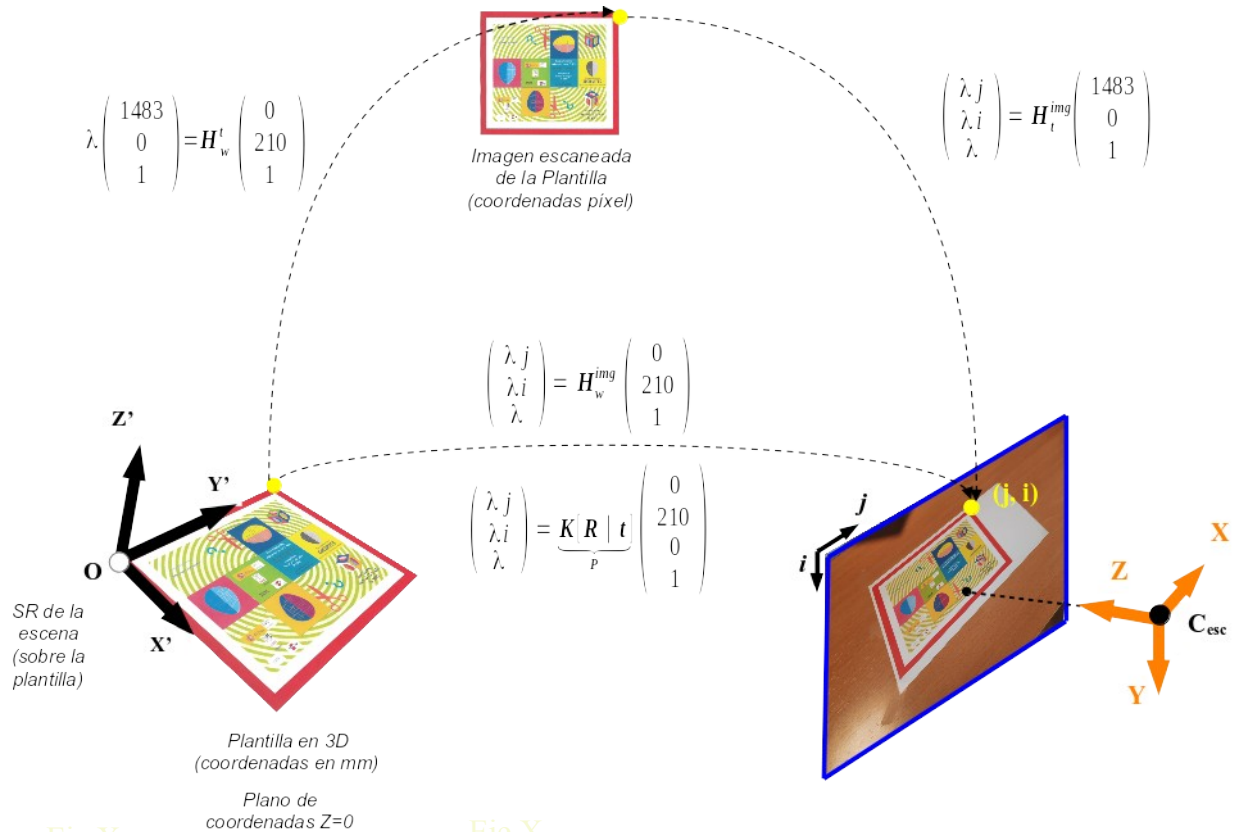


Figura 5: Detalle de la geometría del problema a resolver con las diferentes transformaciones desde el objeto real plano (coordenadas en mm) hasta la imagen de entrada (coordenadas píxel) pasando por la imagen de la plantilla escaneada (coordenadas píxel).

2.1.1 Cálculo de la homografía H_t^{img}

Para el cálculo de la homografía que va desde la imagen de la plantilla escaneada a la imagen de entrada habrá que utilizar detección de puntos de interés y emparejamiento de los mismos mediante comparación de descriptores. Este primer paso nos dará una serie de emparejamientos de puntos

tentativos (muchos pueden ser incorrectos). Utilizando `cv2.findHomography` con RANSAC podremos calcular la homografía buscada y eliminar los emparejamientos incorrectos. La Figura 6 muestra visualmente el uso de esta aproximación.

Es importante darse cuenta que la imagen de la plantilla ya se encuentra recortada para que el píxel (0,0) sea precisamente el que se corresponde con el origen de coordenadas en el objeto real.

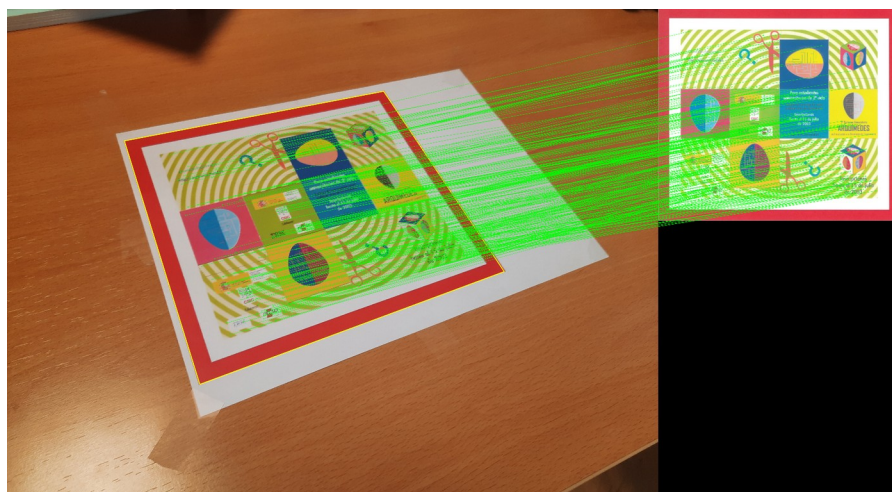


Figura 6: Correspondencias correctas obtenidas después de utilizar `cv2.findHomography`, en amarillo se muestra la transformación de las 4 esquinas de la plantilla escaneada hacia la imagen de entrada, en verde las correspondencias correctas (inliers).

2.1.2 Cálculo de la homografía $\mathbf{H}_w^{\text{img}}$

Una vez tenemos calculada $\mathbf{H}_t^{\text{img}}$ tendremos que calcular $\mathbf{H}_w^t$, esta transformación nos lleva de coordenadas en mm de las cuatro esquinas del objeto real a las coordenadas en píxeles sobre la imagen de la plantilla escaneada. Para ello habrá que pasar las coordenadas de 4 puntos sobre el objeto real y 4 puntos correspondientes sobre la imagen escaneada a `cv2.getPerspectiveTransform`. Para que los pasos posteriores funcionen, habrá que utilizar las 4 esquinas del exterior del marco rojo de la plantilla de las que es fácil encontrar las coordenadas.

Una vez hemos encontrado $\mathbf{H}_w^t$, la homografía que va de puntos sobre el objeto real en mm a la imagen de entrada en píxeles, vendrá dada por: $\mathbf{H}_w^{\text{img}} = \mathbf{H}_t^{\text{img}} \cdot \mathbf{H}_w^t$

2.1.3 Cálculo de $\mathbf{R}$ y $\mathbf{t}$ a partir de $\mathbf{H}_w^{\text{img}}$

En el material de la práctica se proporciona el fichero de texto con la matriz de intrínsecos, $\mathbf{K}$, de la cámara con la que se tomaron las imágenes. La $\mathbf{R}$ y $\mathbf{t}$ de la cámara con respecto al sistema de referencia colocado sobre el plano real (el mostrado en la Figura 4) se puede calcular a partir de $\mathbf{H}_w^{\text{img}}$ como se explica a continuación:

1. Calculamos $\mathbf{H}^* = \mathbf{K}^{-1} \mathbf{H}_w^{\text{img}}$

2. Si $\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3$ son las columnas de la matriz $\mathbf{H}^*$. Entonces tendremos que $\mathbf{h}_1 = \lambda \mathbf{r}_1$, $\mathbf{h}_2 = \lambda \mathbf{r}_2$, y $\mathbf{h}_3 = \lambda \mathbf{t}$ donde $\mathbf{r}_1$ y $\mathbf{r}_2$ son las dos primeras columnas de la matriz $\mathbf{R}$ buscada y $\mathbf{t}$ es el vector de traslación buscado. Así que $\lambda = \|\mathbf{h}_1\| = \|\mathbf{h}_2\|$ con lo que podremos encontrar el valor de $\mathbf{r}_1$, $\mathbf{r}_2$ y $\mathbf{t}$.
3. Para encontrar la tercera columna de $\mathbf{R}$ bastará hacer el producto vectorial $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$.

La matriz de proyección de la cámara será por tanto $\mathbf{P} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}]$ (y en python con numpy $\mathbf{P} = \mathbf{K} @ \text{np.hstack}((\mathbf{R}, \mathbf{t}))$).

2.1.4 Realidad aumentada sobre la imagen

Una vez tenemos la matriz de proyección de la cámara **se pide** pintar el sistema de referencia de la escena (el que hemos definido sobre el plano) proyectando con $\mathbf{P}$ dos puntos sobre cada eje X, Y, Z y uniendo ambos con una línea del color adecuado (rojo, verde y azul, respectivamente para el X, Y y Z).

Lo siguiente que **se pide** en la práctica es colocar un cubo 3D de lado 43 mm descansando sobre el objeto 3D real (uno de los lados tocando el plano $Z=0$) ocupando todo el cuadrado azul claro entre el verde y el amarillo de la plantilla (ver Figura 2). Para hacer esto se proporcionan dos elementos como ayuda:

- Un fichero cubo.obj con los vértices 3D y triángulos que conforman las caras del cubo en formato obj de Wavefront. Será necesario escalar y trasladar convenientemente el cubo para que se encuentre en la zona de la plantilla pedida.
- Una clase Model3D que permite cargar un modelo 3D en formato obj, escalarlo, trasladarlo y mostrarlo proyectado sobre una imagen dada si se dispone de la matriz de proyección $\mathbf{P}$.

Una vez cargado el cubo, escalado y colocado en su sitio sobre la plantilla 3D se podrá calcular la matriz $\mathbf{P}$ para cada imagen de entrada y mostrarlo proyectado sobre la imagen con el método *Model3D.plot_on_image*.

3 Cálculo de $\mathbf{H}_t^{\text{img}}$ con el marco rojo de la plantilla

En este apartado se pide modificar el cálculo de $\mathbf{H}_t^{\text{img}}$ para que se base en la detección de las cuatro esquinas exteriores del marco rojo. Una posible solución pasaría por detectar los píxeles que pertenecen al marco rojo con un umbral en el color, usar la detección de contornos de OpenCV (`cv2.findContours`), simplificar el número de segmentos de cada contorno a 4 (`cv2.approxPolyDP`, algoritmo Ramer–Douglas–Peucker) y quedarse con dos que compartan el mismo centro. Habrá que solucionar problemas como la ordenación de las 4 esquinas para que siempre se devuelvan en el mismo orden y poder calcular la homografía correctamente (aunque la plantilla esté cabeza abajo). En la Figura 7 se muestra un ejemplo en de detección y ordenación correctas de las esquinas del marco rojo de la plantilla.



Figura 7: Esquinas del marco rojo detectadas y ordenadas correctamente.

4 Otras ideas de detección o mejoras

Se valorarán especialmente las prácticas que, además de utilizar el algoritmo establecido en el enunciado, implementen otros detectores que utilicen ideas nuevas o diferentes para calcular la homografía, H_t^{img} , que lleva de la imagen de la plantilla escaneada (*template_cropped.png*) a la imagen de entrada. El nuevo algoritmo de detección propuesto podría no funcionar del todo correctamente pero aún así se valorará si se explica y evalúa razonablemente dejando constancia en la memoria.

Otras ideas de mejora en otras partes de la práctica también se valorarán en este apartado. Por ejemplo, la prueba de diferentes detectores y descriptores de puntos de interés en el apartado 2.1.1.

Cualquier idea de mejora habrá que evaluarla en términos de alguna métrica bien definida. Por ejemplo, la de la distancia promedio en píxeles a la verdadera posición de las cuatro esquinas del marco rojo de la plantilla en la imagen dividido por el perímetro del mismo (no es lo mismo un error de 10 píxeles en un perímetro de 100 que en uno de 1000 píxeles). En este caso habrá que marcar las esquinas con el ratón y guardar las mismas en ficheros de texto como verdadera posición para calcular el error de la detección.

Importante: Solamente se podrán utilizar técnicas de procesamiento de imagen como las vistas en clase. En esta práctica no está permitido utilizar clasificadores (módulo ml de OpenCV) ni otras técnicas parecidas (p.ej. paquetes de python como sklearn, torch, etc).

5 Material proporcionado y formato de salida

Se proporciona una carpeta con los siguientes materiales:

- Carpeta 3d_models con el fichero cubo.obj
- Carpeta imgs_template_real con dos subcarpetas:
 - secuencia: Aquí las imágenes están capturadas con un movimiento pequeño de la cámara entre una y la siguiente.

- test: Aquí las imágenes tienen diferentes orientaciones y posiciones de la cámara con respecto al plano. Se incluyen imágenes donde la el plano está “cabeza abajo”.
- Ficheros python:
 - main.py con el esquema general de la práctica
 - model3d.py con la implementación de la clase Model3D para cargar y dibujar modelos 3D en formato .obj.

Todas las prácticas entregadas deberán:

- Utilizar el script python *main.py* que se proporciona junto con este enunciado.
- La llamada a *main.py* tiene la siguiente estructura:

```
python main.py --test_path <directorio_imágenes> --models_path  
<directorio_modelos_3D> --detector <nombre_detector>
```

- **El primer parámetro** es el directorio donde están las imágenes de test y su correspondiente fichero “intrinsics.txt” (con la matriz K) y fichero “template_cropped.png”
- **El tercer parámetro** es un string con el nombre del detector a ejecutar (por si se implementa más de uno).
- El script main.py deberá crear un directorio “resultado_imgs” desde donde ejecute y en el que se guardarán las imágenes de test procesadas con el sistema de referencia y cubo 3D pintados. Además deberá de mostrar en pantalla estas imágenes con los resultados mientras se procesan.
- El resultado de *main.py* también deberá ser un fichero “resultado.txt” escrito en el directorio donde se ejecute el script, con todas las detecciones para todas las imágenes de test en el mismo, con una línea por cada imagen procesada con las coordenadas de cada uno de los puntos del contorno exterior de la plantilla detectados siguiendo el orden que aparece en la Figura 7 (separado por ;):

```
<nombre_fichero_imagen>; <x0>; <y0>; <x1>; <y1>; <x2>; <y2>; <x3>; <y3>
```

Un ejemplo de detección en la teamplate_cropped.png debería de generar la línea siguiente en el fichero “resultado.txt”:

```
template_cropped.png;0; 0; 1483; 0; 1483; 1306; 0; 1306
```

6 Normas de presentación

La presentación seguirá las siguientes normas:

- Su presentación se realizará a través del Aula Virtual en la fecha del examen.

- Para presentarla se deberá entregar un único fichero ZIP que contendrá el código fuente, el ejecutable y un fichero PDF con la memoria con la descripción del sistema desarrollado.
- La memoria incluirá una explicación con los algoritmos desarrollados, los métodos de OpenCV utilizados, copias de las pantallas correspondientes a la ejecución del programa y discusión del resultado de la ejecución del programa sobre la muestra de test inyendo alguna métrica de rendimiento (p.ej. error promedio en la detección de las 4 esquinas en cada imagen dividido por el perímetro exterior del marco rojo de la plantilla).
- Se permitirán grupos de 2 a 3 alumnos (habrá que registrarse en la tarea de elección de grupo del aula virtual).

La puntuación de esta práctica corresponde al 35% de la asignatura. La práctica se valorará sobre 10, para poder hacer media se necesita al menos un 4,5, y cada parte tendrá la siguiente puntuación:

- Ejercicio 1 (sección 2 - algoritmo de detección básico): **6 puntos.**
- Ejercicio 2 (sección 3 – detección borde rojo de la plantilla): **2,5 puntos.**
- Ejercicio 3 (sección 4 – otras ideas originales o mejoras): **1,5 puntos.**

Para obtener la máxima nota en cada apartado, se valorará:

- Implementar el *main.py* de la manera como se pide en la sección 5.
- La limpieza y organización del código en clases con un Diseño Orientado a Objetos razonable.
- El funcionamiento del código en las imágenes de test.
- Las ideas propias o técnicas pensadas por los alumnos para mejorar lo que se propone en el enunciado.